

25-Day Team Work Distribution and Execution Plan

Secure Token-Efficient Autonomous Coding Agent

Team size: 7

Project lead: Ghulam Qasim

Co-lead / integration lead: Ghulam Qadir

Submission window: 25 calendar days

Operating objective: Complete the project with clear ownership, minimal code collision, controlled integration, and enough time for testing, documentation, and demonstration.

1. Team Structure

1.1 Members and responsibilities

Person	Role	Primary ownership	Secondary responsibility
Ghulam Qasim	Team Lead / Product Owner	Scope, priorities, architecture decisions, acceptance criteria, final submission	Final review, stakeholder communication, conflict resolution
Ghulam Qadir	Co-Lead / Integration Lead	Repository structure, shared contracts, integration branch, CI, release coordination	Code review, daily technical coordination, backup for all modules
Azizullah	Backend and Task-Orchestration Engineer	Task API, run state machine, orchestrator, persistence	API documentation and backend tests
Asif	Security and Policy Engineer	Security Gateway, policy engine, permissions, secret redaction, audit events	Threat model and security test cases
Yash	Sandbox and Tooling Engineer	Tool Router, sandbox runtime, filesystem/shell/network controls	Runtime observability and resource limits
Jami	Context and Model Engineer	Repository index, retrieval/ranking, compression/cache, Nebius adapter/model routing	Token and latency measurements
Aamir	Validation, Evaluation, and Documentation Engineer	Validator/critic, test harness, baselines, metrics, user documentation	Demo preparation and final report

1.2 Management rule

- **Ghulam Qasim decides what must be delivered.**
 - **Ghulam Qadir decides how modules are integrated.**
 - **Module owners decide implementation details inside their owned directories.**
 - No member changes another member's owned module without first creating an issue or obtaining agreement in the team channel.
 - If a decision affects public contracts, security policy, deployment, or the demo, Ghulam Qasim and Ghulam Qadir must approve it.
-

2. Recommended Technology Stack

The stack below is intentionally conservative so the team can finish in 25 days.

2.1 Core stack

Layer	Recommended technology	Reason
Language	Python 3.11+	Fast development, strong testing ecosystem, good AI/API support
API	FastAPI + Pydantic v2	Typed contracts, automatic OpenAPI, async support
Orchestration	Explicit Python state machine	Easy to inspect, test, and demonstrate
Database	PostgreSQL	Reliable run state, policies, approvals, metrics
Local development DB	SQLite only for lightweight unit tests	Avoid blocking local development
Queue	Redis + RQ/Celery, or an in-process MVP queue	Background run execution and cancellation
Cache	Redis	Context summaries, tool results, run locks
Repository index	SQLite/PostgreSQL + tree-sitter or Python AST	Faster MVP than introducing a complex vector platform
Semantic retrieval	Optional sentence-transformer/vector index	Add only after lexical/symbol retrieval works
Model provider	Nebius Token Factory OpenAI-compatible API	Required model integration
Model client	Official OpenAI-compatible Python client	Stable adapter interface
Sandbox	Docker/Podman rootless containers	Filesystem and process isolation
Validation	Pytest, Ruff, Mypy, Bandit where applicable	Tests, quality, type, and security checks

Layer	Recommended technology	Reason
Frontend/demo	Minimal React or server-rendered HTML	Keep UI small; prioritize working backend
Observability	Structured JSON logs + OpenTelemetry-compatible traces	Debugging and evaluation evidence
CI	GitHub Actions or equivalent	Automatic tests and quality checks
Documentation	Markdown + Mermaid diagrams	Version-controlled and easy to review

2.2 MVP scope rule

Do not add Kubernetes, microservices, a vector database, multiple frontend frameworks, or complex agent frameworks unless the basic vertical slice already works. The project is graded on a working, explainable system—not on the number of technologies used.

2.3 Suggested Python dependencies

```
fastapi
uvicorn
pydantic
pydantic-settings
httpx
openai
sqlalchemy
alembic
psycopg[binary]
redis
pytest
pytest-asyncio
ruff
mypy
bandit
python-dotenv
tree-sitter                # only if required for multi-language parsing
tenacity
orjson
opentelemetry-api          # optional but recommended
```

Pin dependency versions in `requirements.txt` or `pyproject.toml` . Do not install new packages directly into the shared environment without discussing the change first.

3. Ownership and Non-Conflict Boundaries

3.1 Directory ownership

The following directory ownership is mandatory. A member may review any directory, but only the owner should be the primary editor.

secure-coding-agent/	
— apps/	
— api/	Azizullah
— orchestrator/	Azizullah
— worker/	Ghulam Qadir + Azizullah
— packages/	
— contracts/	Ghulam Qadir
— task_analyzer/	Azizullah
— security_gateway/	Asif
— prompt_injection/	Asif
— context_optimizer/	Jami
— model_router/	Jami
— tool_router/	Yash
— policy_engine/	Asif
— sandbox_manager/	Yash
— validator/	Aamir
— observability/	Ghulam Qadir
— policies/	Asif
— evaluation/	Aamir
— tests/	
— unit/	Each module owner
— integration/	Ghulam Qadir + Aamir
— security/	Asif + Yash
— docs/	Aamir
— infra/	Yash + Ghulam Qadir
— frontend/	Azizullah, with Qadir integration review

3.2 Shared files requiring approval

Only Ghulam Qadir may merge changes to these files after review from the relevant owners:

```
pyproject.toml
requirements.txt / requirements.lock
.env.example
README.md
packages/contracts/*
Dockerfile / docker-compose.yml
CI workflow files
database migration registry
API route registration files
```

If a member needs a change in a shared file, they should:

1. Open an issue or task note.
2. Explain the required change and affected modules.
3. Let Ghulam Qadir make or coordinate the edit.
4. Run the full relevant test suite before merge.

3.3 Endpoint ownership registry

No two people may implement the same endpoint. The registry below is the source of truth.

Endpoint	Owner	Supporting owner	Status
POST /v1/runs	Azizullah	Qadir	Planned
GET /v1/runs/{run_id}	Azizullah	Qadir	Planned
POST /v1/runs/{run_id}/cancel	Azizullah	Yash	Planned
GET /v1/runs/{run_id}/events	Ghulam Qadir	Aamir	Planned
POST /v1/approvals/{approval_id}/decision	Asif	Azizullah	Planned
GET /v1/runs/{run_id}/diff	Aamir	Azizullah	Planned
GET /v1/runs/{run_id}/validation	Aamir	Azizullah	Planned
GET /v1/health	Ghulam Qadir	Azizullah	Planned
GET /v1/metrics	Ghulam Qadir	Aamir	Planned

Endpoint rule: An endpoint consists of its route, request schema, response schema, service method, database interaction, and tests. One owner is responsible for all five parts.

3.4 Contract-first rule

Before implementation, every cross-module interface must be written in `packages/contracts/` :

- Request and response models.
- Tool request and tool result schemas.
- Policy decision schema.
- Run state values.
- Error codes.
- Event names.
- Version number.

No member should silently alter a shared schema. A breaking change requires:

1. A short change note.
 2. Updated contract tests.
 3. Notification to all affected owners.
 4. Migration or compatibility handling.
 5. Approval from Ghulam Qadir.
-

4. Git and Branching Strategy

4.1 Branches

```
main                # always stable and demo-able
integration          # Qadir's integration branch
feature/<owner>/<task> # one branch per task
hotfix/<issue>        # urgent repair only
```

Examples:

```
feature/azizullah/task-api
feature/asif/policy-engine
feature/yash/tool-router
feature/jami/context-ranking
feature/aamir/validator
```

4.2 Rules that prevent conflicts

1. Never work directly on `main`.
2. Each branch has one clear task and one owner.
3. Keep branches short-lived; target one to three days maximum.
4. Rebase or merge from `integration` before requesting review.
5. Never force-push a shared branch.

6. Do not combine unrelated formatting changes with feature changes.
7. Avoid editing shared files unless the task explicitly requires it.
8. Push at least once per working day.
9. Open a pull request when the task is testable—not only when the whole module is finished.
10. Squash or clean noisy commits before merging.
11. Ghulam Qadir merges into `integration`.
12. Ghulam Qasim approves promotion from `integration` to `main`.

4.3 Commit format

```
<type>(<scope>): <short description>
```

Types: feat, fix, test, refactor, docs, security, infra, chore

Examples:

```
feat(api): add create-run endpoint
security(policy): deny credential paths by default
test validator): add structured pytest failure parsing
docs(plan): document approval workflow
```

4.4 Pull request template

Every PR must include:

```
## Purpose

## Files changed

## Contract/API impact

## Security impact

## Tests executed

## Reviewer requested

## Known limitations
```

4.5 Review matrix

Change area	Required reviewer
API/orchestrator	Qadir + Azizullah
Security/policy	Asif + Yash + Qadir
Sandbox/tool execution	Yash + Asif + Qadir
Context/model	Jami + Aamir + Qadir
Validator/evaluation	Aamir + Qadir
Shared contracts	Qadir + affected module owner
Final release	Qasim + Qadir + all owners

5. Team Operating Rhythm

5.1 Daily schedule

Use one short daily meeting or written stand-up. Maximum 15 minutes.

Each member reports:

1. What I completed yesterday.
2. What I will complete today.
3. What is blocked.
4. Which files or contracts I need from another member.

The stand-up is not for long technical debates. Move debates to a short design note or a separate 20-minute session.

5.2 Daily time blocks

Block	Activity
Start of day	Review task board, contracts, and integration status
Main work block	Individual implementation in owned branch
Midday	Push progress and report blockers
Integration block	Qadir merges tested work and runs CI
End of day	Update task status, test evidence, and next action

5.3 Task board columns

Backlog → Ready → In Progress → Review → Integration → QA → Done

A task cannot enter **In Progress** unless it has:

- One owner.
- A clear acceptance criterion.
- Expected files/directories.
- Dependencies identified.
- A target completion date.

5.4 Conflict-resolution protocol

When members disagree:

1. State the issue in one sentence.
2. Identify whether it affects scope, contract, security, or implementation detail.
3. Prefer the smallest change that preserves the agreed contract.
4. If security is involved, Asif may block the merge until the risk is resolved.
5. If integration is involved, Qadir decides the merge sequence.
6. If scope or deadline is involved, Qasim decides.
7. Record the decision in `docs/decisions.md`.

No disagreement should remain unresolved for more than one working day.

6. Work Distribution by Member

6.1 Ghulam Qasim — Team Lead / Product Owner

Responsibilities

- Freeze MVP scope on Day 1.
- Approve architecture and acceptance criteria.
- Prioritize tasks and remove non-essential features.
- Review progress and resolve scope conflicts.
- Conduct stakeholder communication.
- Approve final release and submission.

Deliverables

- Product scope and definition of done.
- Final feature priority list.
- Acceptance criteria for the demo.
- Final project review checklist.
- Presentation/demo narrative.

Must not do

- Do not become the bottleneck by manually reviewing every file.
- Do not make frequent direct edits inside member-owned modules.
- Do not change technical contracts informally.

Efficient review method

Qasim should review:

- PR summaries and changed-file lists.
- Security decisions and blocked actions.
- CI results and integration status.
- Final API behavior and demo flow.
- High-risk architectural changes.

Qadir performs the detailed code review so Qasim does not need to inspect every line.

6.2 Ghulam Qadir — Co-Lead / Integration Lead

Responsibilities

- Create repository skeleton and branch protections.
- Own shared contracts and integration branch.
- Maintain endpoint registry and dependency map.
- Run daily integration builds.
- Review cross-module changes.
- Coordinate release candidates.
- Maintain CI and shared development scripts.

Deliverables

- Repository layout.
- `packages/contracts/` schemas.
- Integration branch and CI pipeline.

- Run event stream and health endpoints.
- Integration test suite.
- Release checklist.

Authority

Qadir can reject a PR for:

- Contract mismatch.
 - Missing tests.
 - Unclear ownership.
 - Shared-file collision.
 - Unresolved dependency.
 - Broken integration branch.
-

6.3 Azizullah — Backend and Task Orchestration

Owns

```
apps/api/  
apps/orchestrator/  
packages/task_analyzer/  
frontend/ basic task screens
```

Work items

- FastAPI application setup.
- Create-run, status, cancel, and basic run retrieval endpoints.
- Run lifecycle state machine.
- Task input validation.
- Repository/task metadata persistence integration.
- Worker invocation interface.
- Task analyzer and structured task object.

- Backend unit and API tests.

Dependencies

- Contracts from Qadir.
- Policy decisions from Asif.
- Sandbox execution interface from Yash.
- Model worker interface from Jami.

Handoff requirements

Before handing off, provide:

- OpenAPI output.
 - Example request/response payloads.
 - State transition tests.
 - Error behavior.
 - Database migration instructions.
-

6.4 Asif — Security and Policy

Owns

```
packages/security_gateway/  
packages/prompt_injection/  
packages/policy_engine/  
policies/  
tests/security/
```

Work items

- Filesystem allow/deny policy.
- Shell risk classification.
- Network destination policy.

- Git action policy.
- Tool authorization decisions.
- Approval state and high-risk action handling.
- Prompt-injection labeling and detection.
- Secret redaction utilities.
- Audit event definitions.
- Security attack test suite.

Dependencies

- Tool schemas from Qadir.
- Runtime enforcement hooks from Yash.
- Run identity from Azizullah.

Non-negotiable rules

- Security policy must be deterministic for known inputs.
 - Unknown capabilities must default to deny.
 - The model must never be treated as an authorization source.
 - Security tests must run before every release candidate.
-

6.5 Yash — Sandbox and Tooling

Owns

```
packages/tool_router/  
packages/sandbox_manager/  
infra/container/  
infra/runtime/
```

Work items

- Typed tool router.

- Repository read/write tools.
- Structured shell execution.
- Test runner adapter.
- Git read-only adapter.
- Rootless sandbox/container setup.
- Mount restrictions and path canonicalization hooks.
- CPU, memory, disk, process, and timeout limits.
- Runtime output capture.
- Network proxy integration point.

Dependencies

- Policy decisions from Asif.
- Tool contracts from Qadir.
- Run lifecycle from Azizullah.

Handoff requirements

Demonstrate that the runtime blocks:

- `~/ .ssh` and credential paths.
 - `.. /` path traversal.
 - Unrelated repositories.
 - Dangerous commands.
 - Unapproved network destinations.
 - Host process or filesystem access.
-

6.6 Jami — Context and Model Engineering

Owns

```
packages/context_optimizer/  
packages/model_router/
```

Work items

- Repository file inventory.
- Symbol/import/reference index.
- Relevance scoring.
- Context budget allocation.
- Context compression.
- Context cache keys and invalidation.
- Tool-result summarization.
- Loop detection record and threshold logic.
- Nebius OpenAI-compatible adapter.
- Model selection and parameter configuration.
- Token and latency measurement.

Dependencies

- Task object from Azizullah.
- Tool result schema from Qadir.
- Runtime output from Yash.
- Evaluation tasks and metrics from Aamir.

Handoff requirements

Provide comparison evidence for:

- Full repository context vs indexed context.
- Uncompressed output vs compressed output.

- At least two models or two routing configurations.
 - Token count, latency, tool calls, and task success.
-

6.7 Aamir — Validation, Evaluation, and Documentation

Owns

```
packages/validator/  
evaluation/  
docs/  
tests/integration/
```

Work items

- Patch/diff validator.
- Syntax, formatting, lint, and test validation pipeline.
- Structured test failure parser.
- Critic result schema.
- Functional task suite.
- Security evaluation suite in coordination with Asif and Yash.
- Baseline A/B/C experiment harness.
- Metrics report.
- User guide, setup guide, architecture summary, and final report.
- Demo script and expected outputs.

Dependencies

- Run results from Azizullah.
- Policy/audit results from Asif.
- Sandbox output from Yash.
- Context/model metrics from Jami.

Handoff requirements

Every validation report must show:

- What was tested.
 - Exact command.
 - Pass/fail result.
 - Relevant failure evidence.
 - Security findings.
 - Recommendation.
-

7. 25-Day Delivery Schedule

The schedule is divided into five phases. Every phase ends with a review gate. The final five days are protected for integration, bug fixing, documentation, and submission; new major features are prohibited during that period.

Phase overview

Phase	Days	Goal	Gate
Phase 1	1–4	Scope, contracts, skeleton, environment	Architecture and contract freeze
Phase 2	5–10	Build independent modules	Each module has unit tests
Phase 3	11–16	First vertical integration	End-to-end happy path works
Phase 4	17–20	Security, evaluation, and optimization	Release candidate 1
Phase 5	21–25	Stabilize, document, rehearse, submit	Final release

Day-by-Day Plan

Day 1 — Kickoff and scope freeze

All members

- Review architecture and this work distribution.
- Confirm the MVP feature list.
- Confirm what is explicitly out of scope.
- Install the agreed toolchain.
- Create communication channel and task board.

Qasim

- Approve the definition of done.
- Select the demonstration scenario.

Qadir

- Create repository, branches, owners, issue labels, PR template, and basic CI.

Deliverable: approved scope, repository, branch strategy, task board.

Day 2 — Contracts and skeleton

Qadir

- Create contract package and initial event/state schemas.
- Publish API and tool contract draft.

Azizullah

- Create FastAPI and orchestrator skeleton.

Asif

- Create policy format and decision schema.

Yash

- Create sandbox/tool interfaces with mock implementations.

Jami

- Create context/model interfaces with mock provider.

Aamir

- Create validator interfaces, test layout, and evaluation record schema.

Deliverable: all modules import successfully and contracts are versioned.

Day 3 — Local developer workflow

Qadir

- Complete CI: lint, type check, unit tests.
- Add pre-commit or documented local commands.

All members

- Add one passing unit test in their module.
- Confirm branch and PR workflow.

Deliverable: clean CI on `integration`.

Day 4 — Contract freeze 1

All members

- Review schemas, endpoint ownership, tool names, state values, and errors.

Qasim/Qadir

- Approve contract freeze for Phase 2.

Rule: After Day 4, no breaking contract changes without a written decision.

Day 5 — Backend and policy foundations

Azizullah: task models, create-run request, run persistence.

Asif: filesystem rules, default-deny evaluator.

Yash: tool request validation and mock tool executor.

Jami: repository file scanner and task-context interface.

Aamir: validator skeleton and test result schema.

Qadir: merge foundations and resolve contract issues.

Day 6 — Core implementations

Azizullah: run state transitions.

Asif: shell risk classes and approval decision schema.

Yash: repository read/list tools and path validation.

Jami: file metadata index.

Aamir: patch/diff validation and test command abstraction.

Qadir: integration test harness skeleton.

Day 7 — Execution boundaries

Azizullah: worker invocation interface.

Asif: network/Git policies.

Yash: sandbox image and restricted mounts.

Jami: symbol extraction and relevance ranking v1.

Aamir: structured test output parser.

Qadir: CI integration and dependency review.

Day 8 — Model and tool loop interfaces

Azizullah: connect orchestrator to worker interface.

Asif: policy middleware.

Yash: shell/test tool adapters in sandbox.

Jami: Nebius adapter with mocked fallback.

Aamir: validator-to-orchestrator result contract.

Qadir: review all cross-module interfaces.

Day 9 — Unit-test completion

Each member writes unit tests for their primary module and resolves review comments. No new major feature work unless a critical foundation is missing.

Day 10 — Phase 2 gate

Required:

- Each module has a working implementation or explicit stub.
- All stubs are documented.
- CI is green.

- No unowned endpoint or shared-file conflict exists.
- Integration branch can start the application.

Decision: Continue to integration, reduce scope, or remove unstable features.

Day 11 — First vertical slice

Target flow:

```
POST /v1/runs
  → task analysis
  → context selection
  → mocked model response
  → repo read/tool request
  → policy decision
  → sandbox execution
  → validation
  → run result
```

Qadir coordinates. All members fix only the failing boundary they own.

Day 12 — Real repository reading and context

- Index a sample repository.
- Retrieve relevant files.
- Read files through the Tool Router.
- Display provenance and token count.
- Test cache hit and cache miss behavior.

Day 13 — Controlled writing and testing

- Apply a safe patch through `repo.write`.
- Run an approved test command.
- Capture structured test result.
- Produce a diff.

Day 14 — Real model integration

- Connect approved Nebius model configuration.
- Add timeout and retry rules.
- Ensure no secrets enter logs or prompts.
- Run a small fixed task set.

Day 15 — End-to-end happy path

The team must demonstrate one complete successful coding task from request to validated patch. Capture a screen recording or trace for the final presentation.

Day 16 — Integration gate

Required:

- At least one successful end-to-end task.
 - At least one intentional policy denial.
 - Test results are structured.
 - Final diff is available.
 - Run status is visible.
 - Integration branch is reproducible from a clean setup.
-

Day 17 — Security hardening

Asif + Yash lead

Test and fix:

- Credential path access.
- Path traversal and symlink escape.
- Dangerous shell commands.
- Unapproved network requests.
- Unauthorized Git operations.
- Prompt injection in README and comments.
- Secret leakage in logs and model context.

Day 18 — Evaluation and baselines

Aamir + Jami lead

Run:

- Baseline A: basic agent.
- Baseline B: agent plus security.
- Baseline C: agent plus token optimizer.
- Proposed system: security plus token optimizer.

Record tokens, success, latency, tool calls, retries, cost estimate, and blocked actions.

Day 19 — Reliability and failure testing

Test:

- Model timeout.
- Malformed tool call.
- Provider failure.
- Repeated unsuccessful action.
- Test failure and controlled retry.
- Approval timeout.
- Resource exhaustion.
- Cancelled run.

Day 20 — Release Candidate 1 gate

Freeze new feature work except critical defects. Create `release-candidate-1` and execute the full test/evaluation suite.

Day 21 — Bug fixing only

Prioritize by severity:

1. Security vulnerability.
2. Data loss or incorrect repository mutation.
3. Broken end-to-end flow.
4. Failed validation.

5. Incorrect metrics.
6. Documentation or UI issue.
7. Cosmetic improvement.

Day 22 — Documentation and demo

Aamir leads:

- Setup guide.
- Architecture explanation.
- API examples.
- Security explanation.
- Evaluation results.
- Limitations and future work.

Qasim approves the story and presentation order.

Day 23 — Final integration and clean-environment test

- Clone or copy the project into a clean environment.
- Install dependencies from the lock/pinned file.
- Run all tests.
- Run the demo using only documented commands.
- Fix reproducibility problems.

Day 24 — Final rehearsal and submission package

- Full demo rehearsal.
- Time the presentation.
- Prepare screenshots/traces/results.
- Confirm all names and contributions.
- Create final tag and backup archive.
- No experimental feature merges.

Day 25 — Final verification and submission

Qasim: final acceptance.

Qadir: final branch/tag and CI evidence.

Asif/Yash: final security checklist.

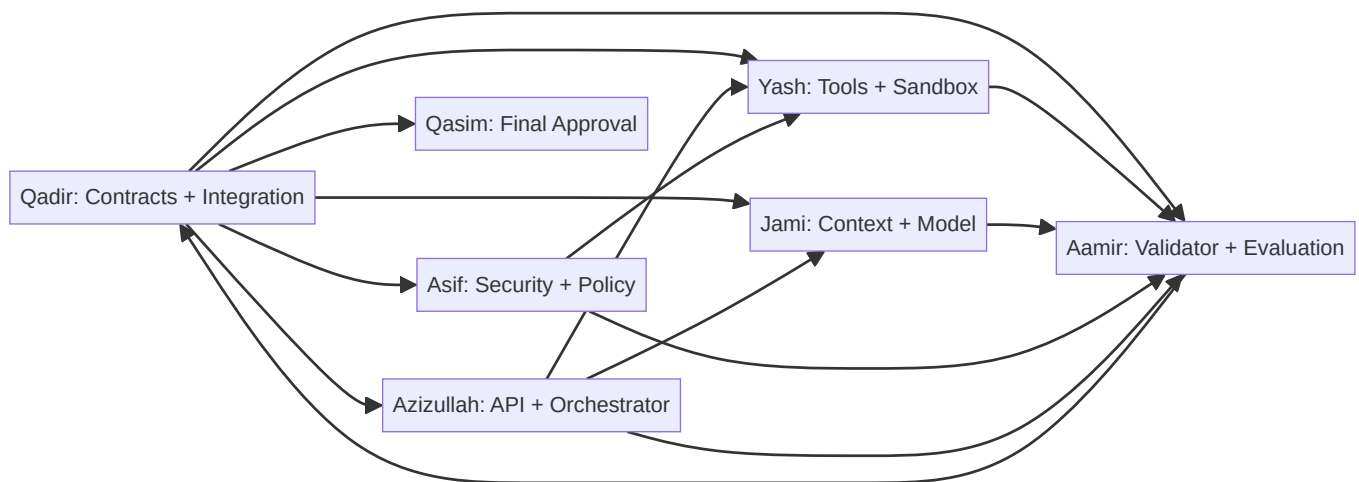
Jami: final model/token evidence.

Aamir: final report and evaluation tables.

Azizullah: final API/demo flow.

Submit only after the final checklist below is marked complete.

8. Dependency and Handoff Map



Handoff format

Every handoff should be a short Markdown note containing:

```
Module:
Owner:
Current branch/commit:
What works:
How to run it:
Inputs:
Outputs:
Known limitations:
Next owner action:
Required files:
```

A handoff is not complete until the receiving owner runs the documented command successfully.

9. Definition of Done

Feature-level done

A feature is done only when:

- The owner implementation is complete.
- Contract is documented.
- Unit tests pass.
- Relevant integration test passes.
- Security impact is reviewed.
- Logs/errors are understandable.
- Documentation or usage example exists.
- PR is reviewed and merged into `integration`.

Module-level done

- Module can be run from a clean checkout.
- No hidden environment variables except documented secrets.
- No direct access to another owner's internal code.
- Public interfaces are typed.
- Failure paths are tested.
- Metrics/events are emitted where relevant.

Project-level done

- One complete coding task works.
- One malicious task is blocked.
- One failed task stops or retries in a controlled way.
- Token optimization has measured evidence.
- Security and functional baselines are recorded.
- Demo can be executed using the README.

- Final tag and backup exist.
-

10. Final Acceptance Checklist

Product

- ☐ User can create a run.
- ☐ User can view run status.
- ☐ Agent analyzes the task.
- ☐ Relevant repository context is retrieved.
- ☐ Model is invoked through the adapter.
- ☐ Tool request is typed and policy checked.
- ☐ Code can be changed in the sandbox.
- ☐ Tests run and results are structured.
- ☐ Diff and final result are returned.

Security

- ☐ Repository is the only writable project area.
- ☐ SSH/cloud credentials are inaccessible.
- ☐ Dangerous commands are blocked.
- ☐ Network is denied by default.
- ☐ Unapproved Git push is blocked.
- ☐ Prompt injection is detected or safely constrained.
- ☐ Secrets are redacted.
- ☐ All policy decisions are audited.
- ☐ Sandbox resource limits work.

Efficiency

- ☐ Full repository is not sent by default.
- ☐ File/symbol ranking works.
- ☐ Context compression works.

- ☐ Cache behavior is measured.
- ☐ Loop detection works.
- ☐ Token usage is recorded.
- ☐ At least two model/configuration options are compared.

Quality and submission

- ☐ Unit tests pass.
 - ☐ Integration tests pass.
 - ☐ Security tests pass.
 - ☐ Clean-environment setup works.
 - ☐ Documentation is complete.
 - ☐ Demo is rehearsed.
 - ☐ Final report includes limitations.
 - ☐ Final tag/backup is created.
-

11. How the Team Lead Avoids Reviewing Every Line

Because Ghulam Qasim cannot inspect the entire codebase, the process should create **review evidence** rather than rely on manual line-by-line review.

Qadir's integration dashboard should show

- Current branch and commit.
- Passing/failing CI status.
- Open PRs by owner.
- Files changed per PR.
- Contract changes.
- Security-sensitive changes.
- Test count and failures.
- Integration blockers.
- Release-candidate status.

Qasim should personally inspect

- Scope and acceptance criteria.
- Architecture and contract changes.
- Security exceptions.
- End-to-end behavior.
- Evaluation evidence.
- Final diff and demo.

Qasim should delegate

- Detailed code review to Qadir and the relevant module owner.
- Security review to Asif and Yash.
- Test/evaluation review to Aamir.
- Token/model review to Jami.

This creates a two-level review structure without making the team lead a bottleneck.

12. Scope Control: What to Cut First

If the team falls behind, cut features in this order:

1. Fancy frontend features.
2. Semantic/vector retrieval.
3. Multiple model routing beyond two configurations.
4. Automatic package installation.
5. Advanced Git operations.
6. Multi-language parsing beyond the main language.
7. Distributed deployment.
8. Non-essential analytics dashboards.

Do **not** cut:

- Policy checks.
- Sandbox isolation.

- Secret redaction.
- Tool validation.
- Test execution.
- Audit logs.
- Basic context optimization.
- Baseline evaluation.

A smaller secure working system is better than a larger system that cannot be demonstrated reliably.

13. Communication Templates

Daily update

```
Name:
Yesterday:
Today:
Blocked by:
Files/modules touched:
PR/commit:
Risk or contract change:
```

Blocker request

```
BLOCKER: <one-line description>
Owner needed: <name>
Expected impact: <task/module>
What I tried:
Decision needed by:
```

Contract-change request

CONTRACT CHANGE REQUEST

Current contract:

Proposed contract:

Why required:

Affected owners:

Backward compatibility:

Tests to update:

Approver: Qadir / Qasim if scope changes

End-of-day integration note

Integration status: GREEN / YELLOW / RED

Merged today:

Tests passed:

New failures:

Security concerns:

Tomorrow's integration target:

14. Final Operating Agreement

The team agrees to the following:

1. We work from the same contract and task board.
2. Every task has one owner.
3. Every endpoint has one owner.
4. Every shared-file change is coordinated through Qadir.
5. We keep branches short and merge tested work frequently.
6. We communicate blockers early rather than hiding them.
7. We do not add features during the final stabilization period.
8. Security decisions override convenience.
9. The model is never allowed to bypass the Policy Engine.

10. We measure our claims instead of making unsupported claims.
11. We protect the final five days for integration and submission.
12. The final release is accepted by Qasim only after the checklist is complete.

Team principle: One owner per component, one source of truth per contract, one integration branch, and one final acceptance decision.